PawSafe

Design Document — Version 2

BIL 481 — Software Engineering
Completion Date: March 29, 2026

List of Contributors

#	Name	Role
1	Yusuf Yanki Utgun	Team Lead / Frontend Dev
2	Eren Can Donertaş	Backend Developer
3	Enes Kerem Goksu	Full Stack / QA
4	Yigit Eren Yilmaz	UI Designer / Tester

Table of Contents

Task Matrix

The following matrix shows the contribution of each team member to the sections of this document.

Section	Utgun	Donertaş	Goksu	Yilmaz
System Overview & Architecture	Lead	Review	Contribute	Review
Software Architecture (Layered)	Lead	Contribute	Review	Review
Design Pattern 1 (Observer)	Contribute	Lead	Review	Review
Design Pattern 2 (Facade)	Review	Review	Lead	Contribute
Updated UML Diagrams	Lead	Contribute	Contribute	Review
Implementation Details	Contribute	Lead	Contribute	Review
Use Case Support in Design	Review	Contribute	Lead	Contribute
Design Decisions	Contribute	Review	Contribute	Lead
Product Backlog & Sprint Planning	Lead	Contribute	Review	Contribute

1. System Overview

1.1 Brief Project Description

PawSafe is a community-driven web application designed to improve public safety by enabling citizens to report and track the locations of dangerous stray animals. Users can interact with an interactive map powered by Leaflet.js to place location markers, attach photo evidence, and validate or dispute reports submitted by others. The platform fosters transparency through a public trust-score leaderboard and includes an admin panel for moderation.

This document (Design Document v2) is an updated version of Design Document v1. It formally establishes the software architecture, introduces two design patterns applied to the system, updates all UML diagrams accordingly, and includes an updated Product Backlog with sprint planning.

1.2 Technology Stack

Layer	Technology	Purpose
Frontend	TypeScript + HTML/CSS	SPA logic, UI rendering
Map Library	Leaflet.js 1.x	Interactive map, markers, popups
Backend	Node.js + Express (TypeScript)	REST API, business logic
Database	PostgreSQL	Persistent data storage
Image Storage	Local filesystem / AWS S3	User-uploaded images
Version Control	Git / GitHub	Source and document management

2. Software Architecture

2.1 Architecture Selection: Layered Architecture

The PawSafe system uses a Layered (N-Tier) Architecture organized into three primary layers:

- Presentation Layer — All user interface components: MapView, NotificationPanel, MarkerList, LeaderboardView, AdminPanel.
- Business Logic Layer — Services that implement core application rules: MarkerService, ValidationService, TrustScoreService, NotificationService, LeaderboardService, AuthService, ImageUploadService.
- Data Access Layer — Repositories that handle all database interactions: MarkerRepository, UserRepository, ImageRepository, ValidationRepository.

2.2 Justification

The Layered Architecture was chosen for the following reasons:

- Separation of Concerns: Each layer has a clear responsibility, making the codebase easier to understand and maintain for a student team.
- Testability: Business logic in the service layer can be unit-tested independently of the UI and the database.
- Flexibility: The Data Access Layer can be swapped (e.g., from local storage to a cloud database) without affecting upper layers.
- Familiar Pattern: The team has prior experience with layered designs, reducing the learning curve and risk of architectural mistakes.
- Appropriate Scale: PawSafe is a medium-scale web application that does not yet require the complexity of microservices or event-driven architecture.

An alternative considered was Event-Driven Architecture, which would suit a system with many concurrent real-time events. However, given the team size, timeline, and the fact that most interactions are request-response (not streaming), Layered Architecture was the more practical and maintainable choice.

3. Design Patterns

Two design patterns have been identified and applied to solve specific structural and behavioral problems in PawSafe. Each pattern decision is documented with its application location, impact on UML diagrams, and the quality factors that guided the decision.

3.1 Pattern 1 — Observer Pattern

Observer Pattern Summary	
Problem	When a marker is validated by the community, multiple parts of the system need to react: the reporter's trust score must increase, a notification card must be shown to the reporter, and the leaderboard must be re-ranked. Without a pattern, ValidationService would need to directly call TrustScoreService, NotificationService, and LeaderboardService — creating tight coupling between unrelated services.
Solution	Apply the Observer (Publish-Subscribe) pattern. ValidationService acts as the Subject. TrustScoreObserver, NotificationObserver, and LeaderboardObserver implement the IObserver interface and register themselves at startup. When a validation event fires, the Subject calls notify(), which calls update() on all registered observers automatically.
Location in Architecture	Business Logic Layer. ValidationService (Subject) and all Observer services reside in this layer. The Presentation Layer is not involved in the notification chain — it only receives the final boolean result.
Quality Factor	Maintainability — coupling between services is eliminated. Adding a new reaction to validation (e.g., sending an email alert) requires only creating a new Observer class without modifying ValidationService.
Quality Criterion	Low coupling score: number of direct dependencies of ValidationService on other services = 0 (was 3 before the pattern). This is measurable via static analysis tools such as ESLint dependency-cruiser.
UML Impact	ISubject and IObserver interfaces added to Class Diagram. Realization arrows drawn from ValidationService to ISubject, and from the three observer classes to IObserver. Component Diagram updated with Observer

	tags on service components. Sequence Diagram shows notify() fan-out to all observers.

3.1.1 TypeScript Interface Signatures

The following interfaces define the Observer contract in TypeScript:

```
interface IObserver {   update(event: ValidationEvent): void; }  interface ISubject
{   subscribe(observer: IObserver): void;   unsubscribe(observer: IObserver): void;
notify(): void; }  class ValidationService implements ISubject {   private
observers: IObserver[] = [];   subscribe(o: IObserver) { this.observers.push(o); }
unsubscribe(o: IObserver) {     this.observers = this.observers.filter(x => x !==
o);   }   notify() {     this.observers.forEach(o => o.update(this.lastEvent));   }
validate(markerId: string, userId: string): boolean {     /* persist vote, then
notify */     this.notify();     return true;   } }
```

3.2 Pattern 2 — Facade Pattern

Facade Pattern Summary	
Problem	Leaflet.js exposes a large and complex API (L.map, L.marker, L.layerGroup, L.popup, L.icon, etc.). Multiple UI components (MapView, AdminPanel, MarkerList) all interact with Leaflet directly. This means every component must import and understand Leaflet internals, causing code duplication and tight coupling to the external library. If the map library were ever replaced (e.g., with Mapbox or OpenLayers), every component would need to be rewritten.
Solution	Apply the Facade pattern by introducing a MapFacade class. MapFacade provides a simple, domain-specific API (addMarker, removeMarker, openPopup, fitBounds, onMapClick) that internally uses Leaflet.js. All UI components interact only with MapFacade — they are completely decoupled from Leaflet.
Location in Architecture	The MapFacade sits between the Presentation Layer and the external Leaflet.js library. It is instantiated once and injected into components that need map interaction. It does not belong to any layer — it acts as an adapter/facade at the boundary of the system.
Quality Factor	Maintainability and Portability — the map library can be changed by modifying only

	MapFacade, not all components. Code duplication across components is eliminated.
Quality Criterion	Fan-in count of Leaflet.js API calls from UI components = 0. All Leaflet API calls are confined to MapFacade (verifiable via code search). Switching map provider requires changes only to MapFacade (1 file vs. N files).
UML Impact	MapFacade class added to Class Diagram with its public interface. Component Diagram updated: Presentation Layer components now point to MapFacade, which in turn points to Leaflet.js (external). Deployment Diagram shows Leaflet.js loaded from CDN and used only via MapFacade.

3.2.1 MapFacade Class Signature

```
class MapFacade {   private map: L.Map;   private markerLayer: L.LayerGroup;
constructor(containerId: string) {     this.map = L.map(containerId).setView([39.9,
32.8], 12);
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(this.map);
this.markerLayer = L.layerGroup().addTo(this.map);   }    addMarker(lat: number,
lng: number, data: MarkerData): string { return id; }   removeMarker(id: string):
void { /* remove from markerLayer */ }   openPopup(id: string, content: string):
void { /* bind popup */ }   fitBounds(bounds: L.LatLngBounds): void {
this.map.fitBounds(bounds); }   onMapClick(callback: (lat: number, lng: number) =>
void): void { /* bind */ } }
```

4. Updated UML Diagrams

All four UML diagrams have been updated to reflect the Layered Architecture selection and the two design patterns (Observer and Facade). Changes from v1 are highlighted in the diagram legends.

4.1 Component Diagram

The Component Diagram shows the three layers of the Layered Architecture. Components tagged with Observer or Facade roles indicate where the design patterns are applied.

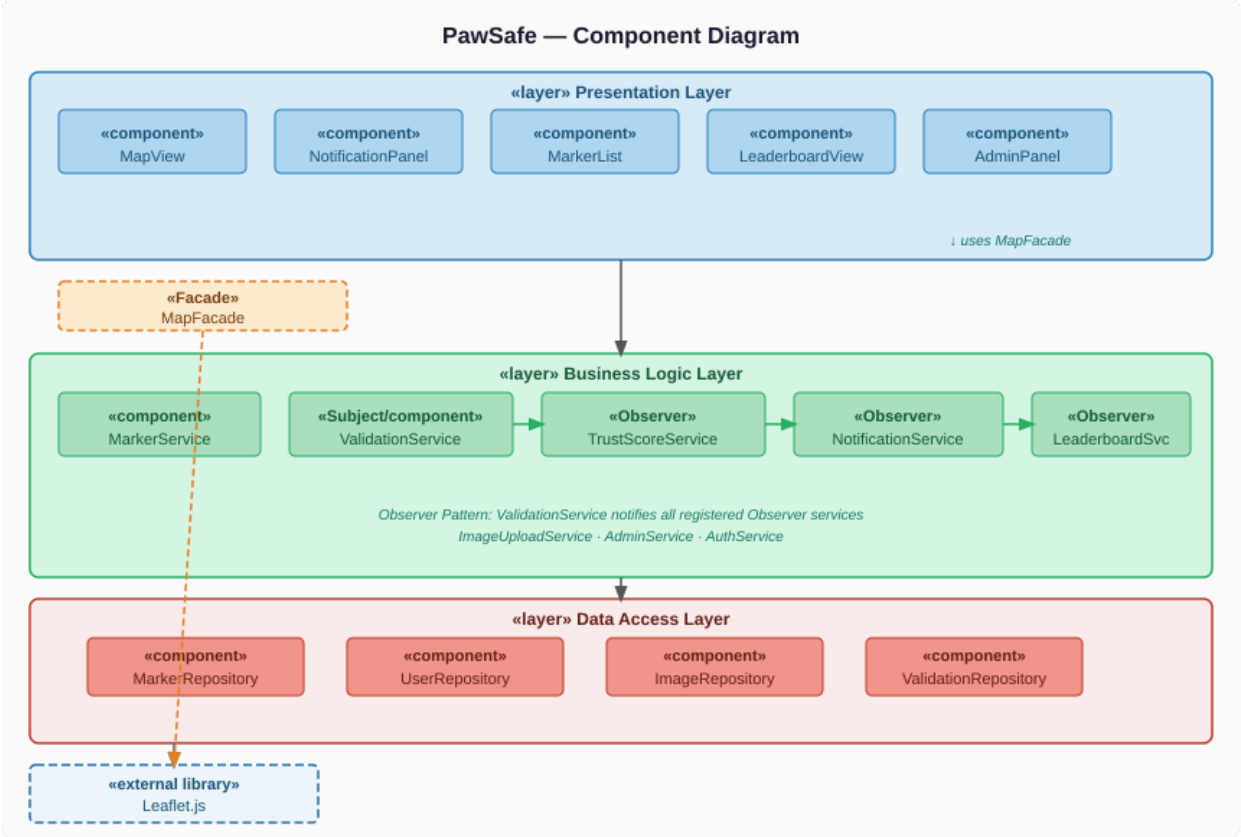

Figure 1 — Component Diagram: Layered Architecture with Observer (Business Logic Layer) and Facade (MapFacade) pattern annotations.

4.2 Class Diagram

The Class Diagram shows the ISubject/IObserver interfaces and their implementations (Observer Pattern), as well as the MapFacade class (Facade Pattern). Method signatures reflect the TypeScript implementation.

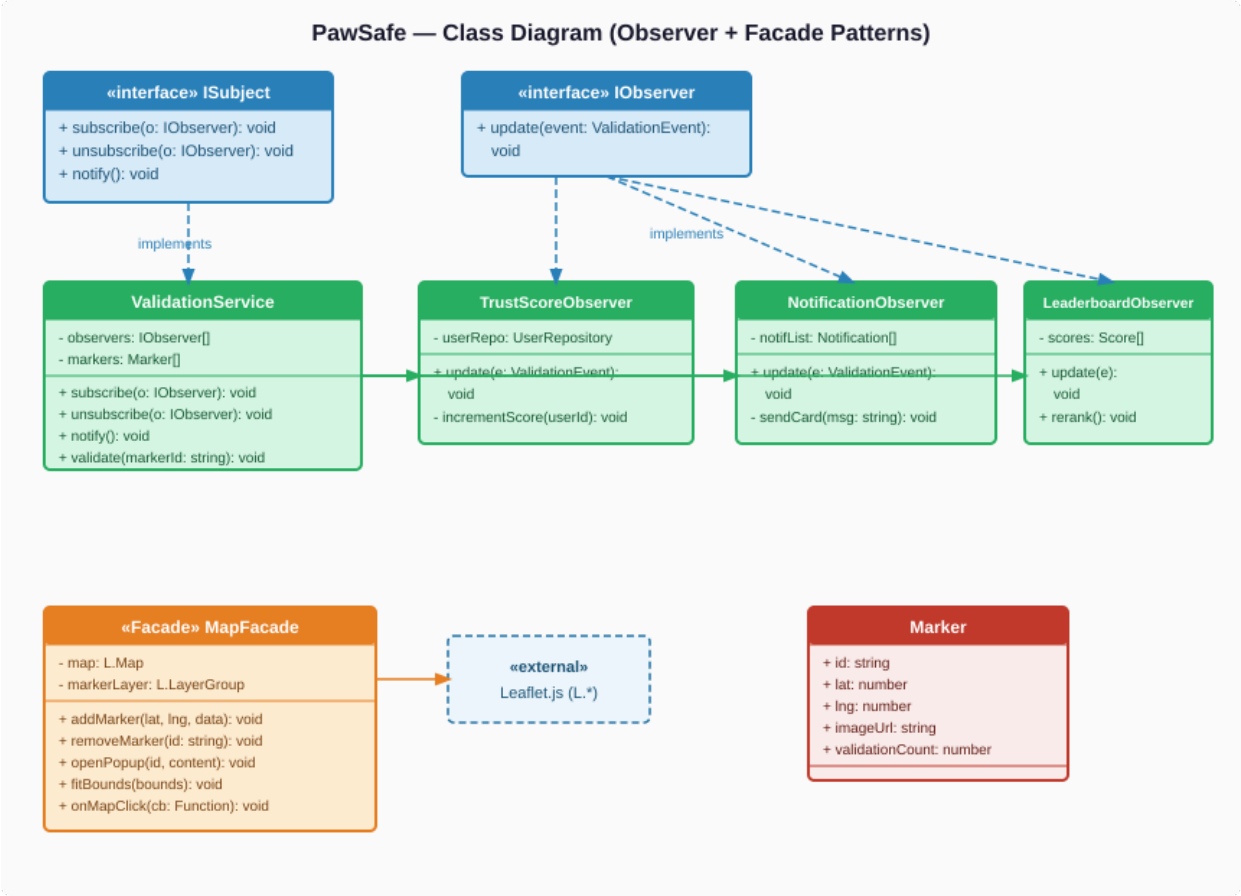

Figure 2 — Class Diagram: Observer Pattern interfaces and Facade Pattern class added to the v1 design.

4.3 Sequence Diagram

The Sequence Diagram illustrates the community validation use case and demonstrates how the Observer Pattern fan-out works at runtime: ValidationService.notify() calls update() on all three observers in sequence.

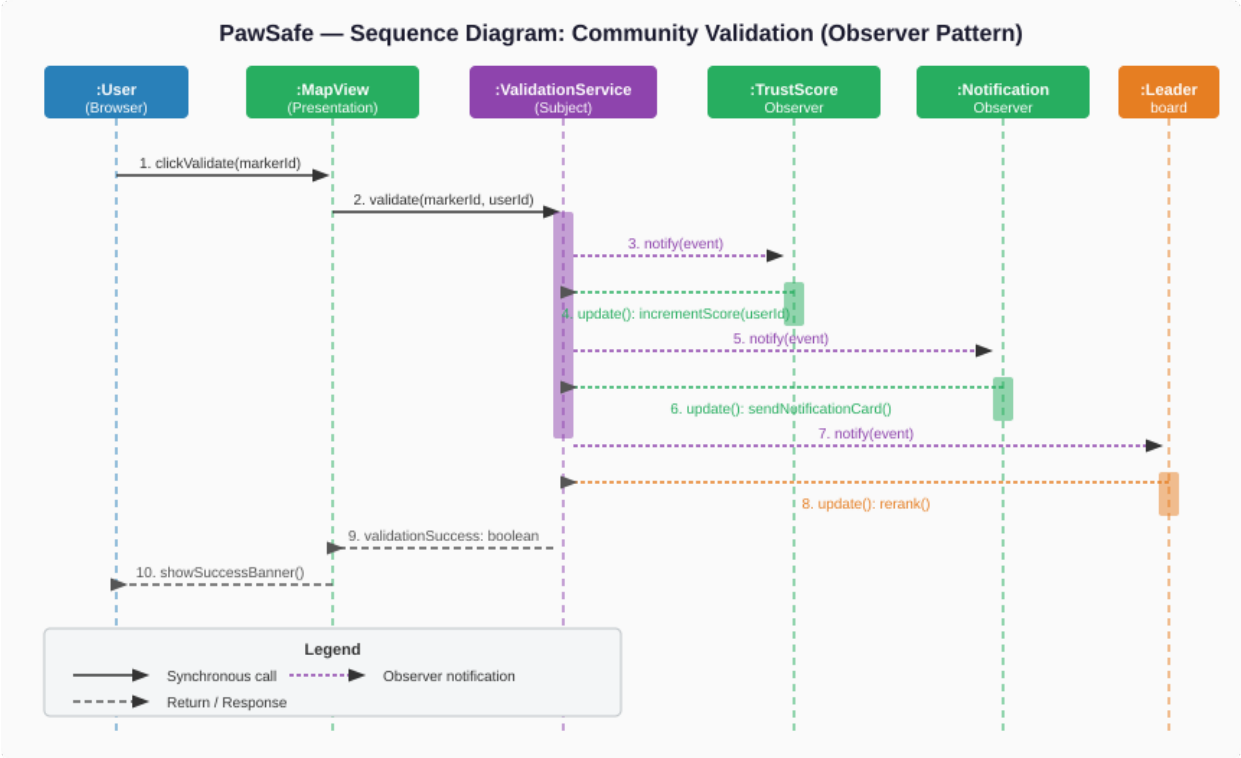

Figure 3 — Sequence Diagram: Community Validation flow using Observer Pattern (notify fan-out highlighted in purple).

4.4 Deployment Diagram

The Deployment Diagram shows the single-server deployment model. MapFacade is noted as the abstraction boundary between the SPA and the Leaflet.js CDN resource.

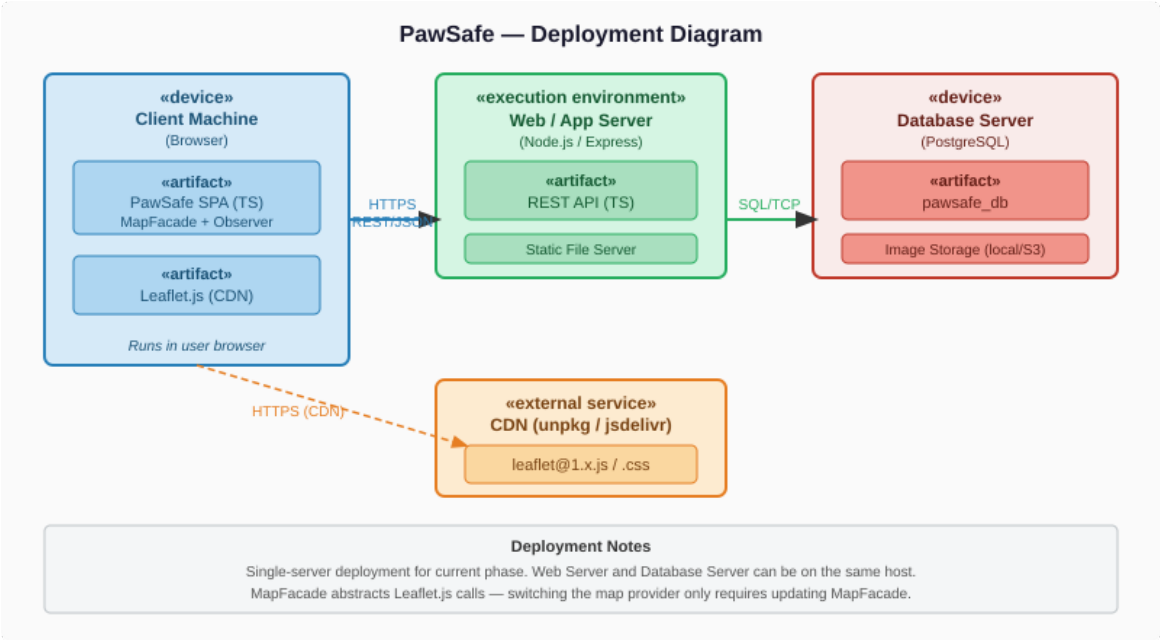

Figure 4 — Deployment Diagram: Single-server deployment. Leaflet.js loaded via CDN, accessed only through MapFacade.

5. Implementation Details

5.1 Codebase Structure

```
pawsafe/   src/     presentation/       MapView.ts       NotificationPanel.ts
LeaderboardView.ts       AdminPanel.ts       MarkerList.ts     business/
ValidationService.ts   # Subject       TrustScoreObserver.ts  # Observer
NotificationObserver.ts       LeaderboardObserver.ts       MarkerService.ts
AuthService.ts       ImageUploadService.ts     data/       MarkerRepository.ts
UserRepository.ts       ImageRepository.ts       ValidationRepository.ts
facade/       MapFacade.ts            # Facade     interfaces/       IObserver.ts
ISubject.ts       types.ts     api/       markers.ts       users.ts
validations.ts   public/   package.json
```

5.2 Key Components and Modules

ValidationService (Subject)

- Manages the list of registered IObserver instances.
- Called when a user submits a validation or dispute for a marker.
- Persists the vote via ValidationRepository, then calls notify() to fan out to all observers.

MapFacade

- Initialized once in MapView and passed as a dependency to other components that need map access.
- Translates domain-level commands (addMarker, removeMarker) into Leaflet.js API calls internally.
- Exposes an onMapClick callback so components can react to map click events without knowing Leaflet.

TrustScoreObserver

- Implements IObserver. On update(), retrieves the reporter's current trust score from UserRepository and increments it by a configurable amount.
- Re-publishes the updated score so LeaderboardObserver can reflect the change.

5.3 Component Interfaces (Method Signatures)

Component	Key Method	Description
MarkerService	createMarker(lat, lng, img, userId): Marker	Creates and persists a new marker
ValidationService	validate(markerId, userId): boolean	Records vote, triggers observer chain
MapFacade	addMarker(lat, lng, data): string	Places marker on Leaflet map, returns id

MapFacade	removeMarker(id: string): void	Removes marker from map layer
TrustScoreObserver	update(event: ValidationEvent): void	Increments reporter trust score
NotificationObserver	update(event: ValidationEvent): void	Pushes in-app notification card
LeaderboardObserver	update(event: ValidationEvent): void	Re-ranks leaderboard entries
AdminService	removeMarker(markerId, adminId): void	Removes marker (admin only)
ImageUploadService	upload(file: File): Promise<string>	Uploads image, returns storage URL

6. Use Case Support in Design

Four use cases are mapped to functional requirements and explained in terms of architectural support.

6.1 UC1 — Marking a Dangerous Location

- Functional Requirements: FR-01 (place marker), FR-03 (attach image), FR-02 (notification card).
- Architecture Support: User clicks map → MapView receives onMapClick event from MapFacade → MapView calls MarkerService.createMarker() → MarkerRepository persists → response triggers NotificationObserver to display confirmation card.
- Design Pattern Involvement: MapFacade abstracts the Leaflet map click event; Observer pattern delivers the confirmation notification.

6.2 UC2 — Community Validation

- Functional Requirements: FR-06 (validate/dispute), FR-07 (trust score update), FR-08 (leaderboard update).
- Architecture Support: User clicks Validate in the Location Info Card → MapView calls ValidationService.validate() → ValidationService persists vote → notify() fires → TrustScoreObserver, NotificationObserver, LeaderboardObserver all update in sequence.
- Design Pattern Involvement: Core use case for the Observer Pattern.

6.3 UC3 — Leaderboard View

- Functional Requirements: FR-07 (trust score), FR-08 (leaderboard).
- Architecture Support: LeaderboardView reads from LeaderboardObserver's maintained score list. The list is kept up-to-date reactively by the Observer chain, so LeaderboardView only needs a simple GET call.
- Design Pattern Involvement: LeaderboardObserver (Observer Pattern) maintains the leaderboard state.

6.4 UC4 — Admin Panel Marker Removal

- Functional Requirements: FR-09 (admin review), FR-10 (remove marker).
- Architecture Support: Admin logs in → AdminPanel is unlocked in Presentation Layer → Admin calls AdminService.removeMarker() → MarkerRepository deletes record → MapFacade.removeMarker() removes the pin from the interactive map.
- Design Pattern Involvement: MapFacade hides Leaflet marker removal complexity from AdminPanel.

7. Design Decisions

7.1 Technology Comparison: Map Libraries

Criterion	Leaflet.js	Google Maps API
Cost	Free, open-source	Paid above usage limit
Customization	High (plugin ecosystem)	Moderate (proprietary)
Bundle size	~40 KB (lightweight)	~200 KB+
TypeScript support	Good (@types/leaflet)	Good (official types)
Decision	Selected	Not selected (cost, size)

7.2 Technology Comparison: Backend Framework

Criterion	Express.js (Node)	Django (Python)
Language	TypeScript (consistent with frontend)	Python (separate language)
Learning curve	Low for the team	Higher for team without Python background
Performance	High (non-blocking I/O)	Good (blocking by default)
Decision	Selected	Not selected (language consistency)

7.3 Justification Summary

- Database: PostgreSQL chosen over MongoDB for relational data integrity (markers -> users -> validations) and familiarity of the team.
- Architecture: Layered over Microservices due to team size, timeline, and appropriate project scale.
- Observer Pattern over direct service calls: eliminates coupling, improves testability.
- Facade Pattern over direct Leaflet usage: reduces coupling to external library, improves portability.

8. Product Backlog and Sprint Planning

8.1 Updated Product Backlog

ID	User Story	Priority	Estimate	Status
PB-01	As a user, I can place a marker on the map at a dangerous animal's location.	High	5 pts	Done
PB-02	As a user, I receive a notification card after placing a marker.	High	3 pts	Done
PB-03	As a user, I can upload an image when placing a marker.	High	5 pts	Done
PB-04	As a user, I can click a marker to view its detail card.	High	3 pts	Done
PB-05	As a user, I can validate or dispute another user's marker.	High	8 pts	In Sprint 2
PB-06	As a user, my trust score increases when my report is validated.	High	5 pts	In Sprint 2
PB-07	As a user, I can see the trust score leaderboard.	Medium	3 pts	In Sprint 2
PB-08	As an admin, I can log in with a dedicated admin account.	High	3 pts	In Sprint 3
PB-09	As an admin, I can remove any marker that is inappropriate.	High	5 pts	In Sprint 3
PB-10	As a user, I can register and log in with a personal account.	High	5 pts	In Sprint 1

8.2 Sprint Planning

Each sprint follows the cycle: Requirement → Design → Implementation → Test.

Sprint 1 Backlog (Weeks 1–2)

Item	Task	Assignee	Status
PB-10	Requirement: User registration & login spec	Donertaş	Done
PB-10	Design: AuthService interface, JWT strategy	Donertaş	Done
PB-10	Implement: register, login, JWT endpoints	Donertaş + Utgun	Done
PB-01	Implement: MapFacade + MapView with Leaflet	Utgun	Done

PB-02	Implement: NotificationPanel component	Yilmaz	Done
PB-03	Implement: ImageUploadService + S3 integration	Goksu	Done
PB-01-03	Test: Unit tests for MarkerService & AuthService	Goksu + Yilmaz	Done

Sprint 2 Backlog (Weeks 3–4)

Item	Task	Assignee	Status
PB-04	Implement: Location Info Card (marker detail popup)	Yilmaz	In Progress
PB-05	Design: Observer Pattern — ValidationService + 3 Observers	Donertaş + Utgun	Done
PB-05	Implement: ValidationService (Subject) with notify()	Donertaş	In Progress
PB-06	Implement: TrustScoreObserver — update trust score on validation	Goksu	In Progress
PB-07	Implement: LeaderboardObserver + LeaderboardView UI	Yilmaz	Planned
PB-05-07	Test: Integration tests — full Observer chain	Goksu + Donertaş	Planned

Sprint 3 Backlog (Weeks 5–6)

Item	Task	Assignee	Status
PB-08	Implement: Admin account role + login flow	Utgun	Planned
PB-09	Implement: AdminPanel UI + AdminService.removeMarker()	Utgun + Yilmaz	Planned
ALL	End-to-end testing of 4 demo use cases	All members	Planned
ALL	Usability testing with external testers	Yilmaz + Goksu	Planned
ALL	Bug fixing and final polish	All members	Planned
ALL	Final documentation and demo preparation	Utgun (lead)	Planned